



Department:	Informatics	Specialization:	Artificial Intelligence
Instructor:	Eng. Noor Al-Hakim	Course Name:	ISA



Sokoban Auto Solver Project Report

Introduction

Sokoban is one of the most well-known puzzle games and is widely used in Artificial Intelligence as a practical example of State Space Search problems. The game consists of a player moving boxes inside a warehouse-like environment surrounded by walls. The objective is to push all boxes onto designated target locations while minimizing the number of moves and respecting the movement constraints of the game.

The complexity of Sokoban arises from the existence of deadlock situations, where one or more boxes become trapped in positions from which the puzzle can no longer be solved. As a result, Sokoban has become a popular benchmark for evaluating search algorithms and heuristic techniques in Artificial Intelligence.

The goal of this project is to develop a complete Sokoban game using Python and the Tkinter graphical library while integrating an automatic solver based on the A* search algorithm. The project focuses on efficient state representation, heuristic-based search, deadlock detection, performance optimization, and interactive graphical visualization.

System Description and Project Architecture

The system follows the Object-Oriented Programming (OOP) paradigm and is divided into three main components:

1. Start Screen

The Start Screen serves as the entry point of the application. It provides a graphical welcome interface containing a background image and a button that launches the game.

This separation improves the user experience and keeps the application structure organized by isolating initialization procedures from gameplay functionality.

2. Game Engine (Sokoban Game)

The game engine is the core component of the application and is responsible for:

- Loading and parsing level data.
- Managing player, box, wall, and target positions.
- Validating movements according to Sokoban rules.
- Executing box-pushing operations.
- Counting player moves.
- Detecting level completion.
- Managing manual keyboard controls.



Department:	Informatics	Specialization:	Artificial Intelligence
Instructor:	Eng. Noor Al-Hakim	Course Name:	ISA



- Rendering graphical elements in real time.
- Recording and displaying level completion time.

The game engine also provides controls for restarting levels, switching between levels, controlling solver execution speed, and displaying gameplay statistics.

3. Automatic Solver System (Sokoban Solver)

The automatic solver is implemented separately from the graphical interface, improving maintainability and scalability.

The solver is responsible for:

- Exploring possible game states.
- Evaluating states using heuristic functions.
- Detecting deadlocks.
- Finding the shortest solution path.
- Returning the sequence of moves required to solve the level automatically.

State Representation and Game Rules

Each game state is represented using the following components:

Player Position

The player position is represented as an ordered coordinate pair:

(x, y)

Box Positions

All box locations are stored as a set of coordinates.

Goal Positions

Target positions are represented as a fixed set of coordinates.

Wall Positions

Walls are stored as non-traversable coordinates.

A state is considered a goal state when every box occupies a target position.

Illegal moves are immediately rejected, including:

- Moving into walls.

Department:	Informatics	Specialization:	Artificial Intelligence
Instructor:	Eng. Noor Al-Hakim	Course Name:	ISA

- Pushing boxes into walls.
- Pushing boxes into other boxes.
- Moving outside the map boundaries.

A* Algorithm Implementation

The automatic solver uses the A* search algorithm, which combines path cost and heuristic estimation.

The evaluation function is defined as:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$: Cost of reaching the current state.
- $h(n)$: Estimated remaining cost to reach the goal.

Heuristic Function

The heuristic function uses Manhattan Distance to estimate the remaining cost.

For each box, the minimum Manhattan distance to the nearest target is calculated and summed.

In addition, a small weighted distance between the player and the nearest box is included to guide the search more effectively.

This heuristic significantly reduces unnecessary exploration and improves solver efficiency.

Search Data Structures

The implementation uses several efficient data structures:

- Priority Queue (Heap) for managing the Open Set.
- Hash-based structures for storing visited states.
- Dictionaries for tracking path costs and reconstructing solutions.

These structures help improve search performance and prevent repeated state exploration.

Search Limits

To prevent excessive memory consumption and long execution times, practical limits are applied during the search process.

The solver terminates when:

- A solution is found.



Department:	Informatics	Specialization:	Artificial Intelligence
Instructor:	Eng. Noor Al-Hakim	Course Name:	ISA



- The maximum number of explored states is reached.
- The maximum allowed search time is exceeded.

These limits ensure stable execution even on difficult levels.

Deadlock Detection

Deadlock detection is one of the most important optimizations in Sokoban.

Several mechanisms were implemented:

Corner Deadlock Detection

Boxes trapped in corners that are not target positions are immediately identified as deadlocks.

Adjacent Box Deadlock Detection

Configurations where neighboring boxes block each other permanently are detected and discarded.

Corridor Deadlock Detection

Additional checks are applied in narrow corridors and complex map structures to avoid exploring unsolvable states.

These techniques significantly reduce the search space and improve solver performance.

Graphical User Interface and User Interaction

The graphical user interface was implemented using the Tkinter library.

External images are used to represent:

- Walls
- Player
- Boxes
- Targets
- Boxes on targets

The interface provides the following features:

- Graphical grid visualization.
- Real-time game updates.
- Keyboard-based movement control.

Department:	Informatics	Specialization:	Artificial Intelligence
Instructor:	Eng. Noor Al-Hakim	Course Name:	ISA

- Restart Level button.
- Previous Level button.
- Next Level button.
- Auto Solve button.
- Stop Auto Solve button.
- Solver speed control slider.
- Move counter.
- Difficulty indicator.
- Level timer.
- Time recording functionality.

The interface provides smooth interaction between the player and the game environment while continuously displaying important gameplay information.

Performance Evaluation and Results

The system was tested on multiple levels with varying difficulty levels.

The experimental results showed that:

- Beginner and Intermediate levels were solved quickly.
- Advanced levels required more search effort.
- Deadlock detection substantially reduced the number of explored states.
- The A* heuristic improved solution quality and search speed.

Sample Results

Level	Difficulty	Solved	Performance
1	Beginner	Yes	Very Fast
2	Intermediate	Yes	Fast
3	Advanced	Yes	Moderate
4	Expert	Yes	Slower
5	Master	Challenging	High Search Cost



Department:	Informatics	Specialization:	Artificial Intelligence
Instructor:	Eng. Noor Al-Hakim	Course Name:	ISA



The built-in timer was also used to compare manual gameplay performance with automatic solver performance.

As expected, highly complex levels required significantly larger search spaces due to the computational complexity of the Sokoban problem.

Conclusion

This project presents a complete implementation of a Sokoban game integrated with an A* based automatic solver. The project combines Artificial Intelligence search techniques, Object-Oriented Programming principles, and interactive graphical design to create an educational and practical software system.

The implemented heuristic functions and deadlock detection mechanisms successfully improved solver efficiency and reduced unnecessary exploration of unsolvable states.

The modular architecture allows future expansion and experimentation with additional search techniques.

Future Work

Possible future improvements include:

- Implementing IDA* (Iterative Deepening A*).
- Bidirectional Search techniques.
- Pattern Database heuristics.
- Reinforcement Learning approaches.
- Additional level generation systems.
- More advanced deadlock detection mechanisms.
- Performance comparison between multiple AI search algorithms.

Overall, the project demonstrates the practical application of Artificial Intelligence search algorithms to a classical planning and optimization problem while providing an intuitive and interactive user experience.